Assignment1 Day6

```
Res_perm = [10, 14, 105, 20, 110, 150, 5, 200, 100, 350,  25, 33,
35 ,40, 315, 0, 45, 290, 80]

res_high_perm = []
res_low_perm = []
def perm_classify(X):
  for i in X:
    if i>= 100:
      res_high_perm.append(i)
    else:
      res_low_perm.append(i)

  print(f'Reservoir High permeability list:  {res_high_perm}')
  print(f'Reservoir Low permeability list: {res_low_perm}')

perm_classify(Res_perm)
```

```
Reservoir High permeability list:  [105, 110, 150, 200, 100, 350, 315,
290]
Reservoir Low permeability list: [10, 14, 20, 5, 25, 33, 35, 40, 0,
45, 80]
```

Assignment 2 - Day 6

```
Sw = list(range(5, 40, 5))
print(Sw)
```

```
[5, 10, 15, 20, 25, 30, 35]
```

```
def So_Calc(a):
  So = []
  for i in a:
    So.append(100-i-10)
  return So

So_Calc(Sw)
```

```
[85, 80, 75, 70, 65, 60, 55]
```

```
PG1 = [10, 20, 30]
PG2 = [20, 10, 5]
PG3 = [5, 10, 5]
PG4 = [5, 10, 15]
PG5 = [5, 5, 5]

avo_class = []
def avo(a):
    index = 0
```

```
    if a[index+1] > a[index] and a[index+2] > a[index+1]:
        avo_class = list(("Class 3",))
    elif a[index+1] < a[index] and a[index+2] < a[index+1]:
        avo_class = list(("Class 1" , ))
    else:
        avo_class = list(("No AVO", ))
    return avo_class

avo(PG1)
```

```
['Class 3']
```

```
avo(PG2)
```

```
['Class 1']
```

```
avo(PG5)
```

```
['No AVO']
```

```
PG = [ [10, 20, 30], [20, 10, 5], [5, 10, 5], [5, 10, 15], [5, 5, 5]]

#List[sublist_Index][item_index]

avo_class = []

def avo(a):
    index = 0
    sub_index = 0
    for index in range(len(a)):
        if a[index][sub_index+1] > a[index][sub_index] and a[index]
[sub_index+2] > a[index][sub_index+1]:
            avo_class.append('Class 3')
        elif a[index][sub_index+1] < a[index][sub_index] and a[index]
[sub_index+2] < a[index][sub_index+1]:
            avo_class.append('class 1')
        else:
            avo_class.append('No AVO')
    return avo_class

avo(PG)
```

```
['Class 3', 'class 1', 'No AVO', 'Class 3', 'No AVO']
```

Tuples

```
#Tuples are immutable

#create
```

```
#tuple = (1, "a", 2.2)
#tuple = tuple((, , , , ))

a = ()
type(a)
```

```
tuple
```

```
Data = "pressure", 4500, "Thickness", 250

type(Data)
```

```
tuple
```

```
Depth = (2500)

type(Depth)
```

```
int
```

```
Depth = (2500, )
type(Depth)
```

```
tuple
```

```
#Tuple indexing   >>> Same as lists

#Tuple_Name[index_position]

#Tuple_Slicing    >>> Same as lists

#Tuple_Name[start_index : End_index : Step_Argument]

Basins = ("SHOUSHAN", "GHAZALAT", "MATRUH", "ABU GHARADIG", "ALAMAIN",
"NATRUN")

type(Basins)
```

```
tuple
```

```
Basins[3] = "ABU-GHARADIG"
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call
last)
Cell In[24], line 1
----> 1 Basins[3] = "ABU-GHARADIG"

TypeError: 'tuple' object does not support item assignment
```

```
Basins.append("GINDI")
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[25], line 1
----> 1 Basins.append("GINDI")

AttributeError: 'tuple' object has no attribute 'append'
```

```
Basins.pop()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[26], line 1
----> 1 Basins.pop()

AttributeError: 'tuple' object has no attribute 'pop'
```

```
Basins.remove("ABU GHARADIG")
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[27], line 1
----> 1 Basins.remove("ABU GHARADIG")

AttributeError: 'tuple' object has no attribute 'remove'
```

```
Other_Basins = ("GINDI", )

Basins.extend(Other_Basins)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[29], line 1
----> 1 Basins.extend(Other_Basins)

AttributeError: 'tuple' object has no attribute 'extend'
```

```
All_Basins = Basins + Other_Basins

All_Basins
```

```
('SHOUSHAN',
 'GHAZALAT',
 'MATRUH',
 'ABU GHARADIG',
```

```
 'ALAMAIN',
 'NATRUN',
 'GINDI')

# in keyword   >>> same as lists

#Count method    >>> same as lists

#.index method   >>> same as lists


Field_Names = ('Ras Fanar', 'Ras Fanar', 'October', 'October',
'Belayim', 'Ras Fanar',
            'Sidri', 'Ras Fanar', 'Morgan', 'October', 'October',
'Sidri', 'Shoab Ali',
            'Zeit Bay', 'Belayim', 'Esh El Mallaha', 'Zeit Bay',
'Shoab Ali', 'Morgan',
            'Zeit Bay', 'Sidri', 'Morgan', 'Shoab Ali', 'Shoab Ali',
'Gemsa', 'Shoab Ali',
            'October', 'Shoab Ali', 'Ras Fanar', 'Zeit Bay')

Field_Names.index("October")

2

Field_Names.index("October", 3)

3

Field_Names.index("October", 4, 8)

---------------------------------------------------------------------------
ValueError                                Traceback (most recent call
last)
Cell In[36], line 1
----> 1 Field_Names.index("October", 4, 8)

ValueError: tuple.index(x): x not in tuple

#While loops   >> same as lists

#For loops   >>> same as list


#Tuple packing : Crate a tuple

#Tuple Unpacking:  extract tuple items into different variables

Area_Info = 90, 5000, "Denise", "Developement", 2020

type(Area_Info)
```

```
tuple

wells , daily_prod, Field, Status, expiry = Area_Info

wells

90

Status

'Developement'

Measured_Depth = 30, 210, 510 , 750, 1250, 1230, 1548, 1750, 1790,
1853, 1910, 2060, 2100
Start_Depth, *Intervals, Target_Depth, total_septh = Measured_Depth

type(Measured_Depth)

tuple

Start_Depth, Total_Depth = Measured_Depth[0], Measured_Depth[-1]

Start_Depth

30

Total_Depth

2100

Start_Depth, *Intervals, Total_Depth = Measured_Depth

Intervals

[210, 510, 750, 1250, 1230, 1548, 1750, 1790, 1853, 1910, 2060]

#Lists inside Tuple

all_Info = 90, 5000, "Field-X", ['Eni', 35, 'bp', 15, 'EGPC',
50],"Development", 2030, True

#No_of_Wells, Daily_production, Field_Name, Sharing_Percentage,
Status, Expiry_Date, On_Production

all_Info[3]

['Eni', 35, 'bp', 15, 'EGPC', 50]

all_Info[3][4] = "EGAS"

print(all_Info)

(90, 5000, 'Field-X', ['Eni', 35, 'bp', 15, 'EGAS', 50],
'Development', 2030, True)
```

```
all_Info[3].pop(2)
```

```
'bp'
```

```
print(all_Info)
```

```
(90, 5000, 'Field-X', ['Eni', 35, 15, 'EGAS', 50], 'Development',
2030, True)
```

```
all_Info[3].insert(2, "BG")
```

```
print(all_Info)
```

```
(90, 5000, 'Field-X', ['Eni', 35, 'BG', 15, 'EGAS', 50],
'Development', 2030, True)
```

```
wells_prod = 300, 500 , 550, 1250, 120, 1480, 750, 190, 185, 190, 260,
1100


Min_prod = min(wells_prod)
print(Min_prod)
```

```
120
```

```
Max_prod = max(wells_prod)
print(Max_prod)
```

```
1480
```

Dictionaries

```
#unordered  >> don't maintain order ,,, can't be accessed using index

# Mutable  >>> can change

#Don't Allow duplicates

# items  KEY (Variable Name) : VALUE (Variable Value)



#Syntax_1 {"key" : value , "key" : value, "key" : value,"key" :
value ,"key" : value }

Res = [ "a", "b", "c"]
phi = [0.25, 0.2, 0.17]

Res_Phi_dict ={"a":0.25, "b":0.2, "c": 0.17}
type(Res_Phi_dict)
```

```
dict
```

```
#Syntax_2  dict(Key=Value, Key=Value, Key=Value)

R_Phi_dict = dict(a=0.25, b=0.2, c=0.17)
type(R_Phi_dict)
```

```
dict
```

```
#Syntax_3   dict[('Key',Value) ,('Key',Value) ,('Key',Value) ]

R_dict = dict([("a", 0.25), ("b", 0.2), ("c", 3)])
type(R_dict)
```

```
dict
```

```
Field_Data = {
    'field_name': 'Abu Rudeis',
    'producing wells':['Well-12','Well-13', 'Well-44', 'Well-43'],
    'status':'Developement',
    'daily production rate': 400, #cubic_meters
    'producing reservoirs': ['Nubia', 'Nukhul', 'Lower Sennonian'],
    'average porosity' : 19 #%
}
```

```
#Access dict.   Dict_name["Key"]

Field_Data["field_name"]
```

```
'Abu Rudeis'
```

```
Field_Data["producing wells"]
```

```
['Well-12', 'Well-13', 'Well-44', 'Well-43']
```

```
Field_Data["producing wells"][1]
```

```
'Well-13'
```

```
Field_1 = {'Reservoir_name': 'A', 'Basin': 'GOS', 'avg_phi': 0.16,
'avg_perm': 120, 'Net pay': 40}

Field_2 = {'Reservoir_name': 'B', 'Basin': 'MED', 'avg_phi': 0.25,
'avg_perm': 180, 'Net pay': 15}

Field_3 = {'Reservoir_name': 'C', 'Basin': 'ND', 'avg_phi': 0.20,
'avg_perm': 200, 'Net pay':25}


Field_1["avg_phi"]
```

```
0.16
```

```
Fields = {'Field_1' : {'Reservoir_name': 'A', 'Basin': 'GOS',
'avg_phi': 0.34, 'avg_perm': 35, 'Net pay': 40},
 'Field_2' : {'Reservoir_name': 'B', 'Basin': 'MED', 'avg_phi': 0.25,
'avg_perm': 28, 'Net pay': 15},
 'Field_3' : {'Reservoir_name': 'C', 'Basin': 'ND', 'avg_phi': 0.29,
'avg_perm': 50, 'Net pay':25}
}
```

```
Fields["Field_2"]
```

```
{'Reservoir_name': 'B',
 'Basin': 'MED',
 'avg_phi': 0.25,
 'avg_perm': 28,
 'Net pay': 15}
```

```
Fields["Field_2"]["Net pay"]
```

```
15
```

```
Fields["Field_1"]["Reservoir_name"]
```

```
'A'
```

```
#adding Data


Field_Dict = {}

Field_Dict["Field"] = "Denise"

Field_Dict["Reservoir"] = "Pliocene"

Field_Dict["Reserve"] = 16 #unit

print(Field_Dict)
```

```
{'Field': 'Denise', 'Reservoir': 'Pliocene', 'Reserve': 16}
```

```
len(Field_Dict)
```

```
3
```

```
#in keyword


drilling_data = {
 'drill_bit' : 'PDC',
 'ROP' : 20,
 'measurements': {'TVDSS' : 1500, 'MD': 2100},     #Dictionary inside
Dictionary
 'lithology' : ['shale', 'limestone', 'sandstone'] #List inside
```

```
Dictionary
}
```

```
"ROP" in drilling_data
```

```
True
```

```
if "ROP" in drilling_data:
    print("Exist")
else:
    print("Doesn't exist")
```

```
Exist
```

```
if "measurements" in drilling_data:
    print("Exist")
else:
    print("Doesn't exist")
```

```
Exist
```

```
if "TD" in drilling_data:
    print("Exist")
else:
    print("Doesn't exist")
```

```
Doesn't exist
```

```
if "PDC" in drilling_data:
    print("Exist")
else:
    print("Doesn't exist")
```

```
Doesn't exist
```

```
#.values()
```

```
if "PDC" in drilling_data.values():
    print("Exist")
else:
    print("Doesn't exist")
```

```
Exist
```

```
print(drilling_data)
```

```
{'drill_bit': 'PDC', 'ROP': 20, 'measurements': {'TVDSS': 1500, 'MD':
2100}, 'lithology': ['shale', 'limestone', 'sandstone']}
```

```
if "limestone" in drilling_data.values():
    print("Exist")
else:
    print("Doesn't exist")
```

```
Doesn't exist
```

```python
if "limestone" in drilling_data["lithology"]:
    print("Exist")
else:
    print("Doesn't exist")
```

```
Exist
```

```python
for i in drilling_data:
    print(i)
```

```
drill_bit
ROP
measurements
lithology
```

```python
for i in drilling_data:
    print(drilling_data[i])
```

```
PDC
20
{'TVDSS': 1500, 'MD': 2100}
['shale', 'limestone', 'sandstone']
```